CSC5034Z 2026 — Assignment 2

Encoder-Decoder Models for Grapheme-to-Phoneme Conversion

Kananelo Chabeli CHBKAN001

May 15, 2026

AI Usage Statement: This work was completed independently in accordance with the academic integrity policies of the University of Cape Town. All experimental design, architectural decisions, result interpretation, and written analysis are my original work. All sources consulted have been appropriately cited in the bibliography.
Artificial Intelligence tools were used in a limited and supplementary capacity during the completion of this assignment, specifically for code debugging, problem clarification, grammatical error correction, and assistance locating academic references. At no point was AI used to generate substantive written analysis, experimental results, or core design decisions submitted as part of this work.

1 Implementation Design

This assignment implements an LSTM-based encoder-decoder for grapheme-to-phoneme (G2P) conversion in three context modes: no context (*bottleneck baseline*), fixed context vector, and dot-product cross-attention.

1.1 LSTM Cell

`LSTMCell` implements the six standard LSTM equations (Appendix A.1) from scratch using `nn.Linear` layers with no built-in PyTorch RNN modules. An `is_decoder` flag activates four additional linear projections for each gate that inject a context vector $\mathbf{z}_t$:

$$f_t = \sigma(W_f\, y_{t-1} + U_f\, h_{t-1} + V_f\, \mathbf{z}_t + b_f)\,, \tag{1}$$

and analogously for the input, output, and candidate-cell gates. When `is_decoder=False` (encoder mode) the context-vector projections are omitted entirely.

`LSTMModel` stacks an arbitrary number of `LSTMCell` layers and processes a full sequence, threading hidden and cell states across time steps and layers. It serves as the common parent for both `Encoder` and `Decoder`, avoiding code duplication.

1.2 Encoder

`Encoder` extends `LSTMModel` with a source embedding (`nn.Embedding`) and overrides `forward` to accept integer token indices of shape (batch, src_len). It embeds, transposes to (src_len, batch, d_e), delegates to the parent, and returns `(all_h, (h_n, c_n))` exactly as specified.

1.3 Decoder and Context Modes

`Decoder` also extends `LSTMModel`. A `context_mode` argument selects one of three behaviours at each decode step:

- `none` — no context; `is_decoder=False` so no context weights are allocated. Encoder information reaches the decoder only through the initial hidden state.
- `fixed` — the encoder's final hidden state $\mathbf{z} = h_n^{\text{enc}}$ is passed to every gate at every decode step through the context-vector projections.
- `atten` — a step-specific context vector $\mathbf{z}_t$ is recomputed at each step via dot-product attention over all encoder hidden states (see Section 1.4).

A linear `head` layer maps the final hidden state to logit scores over the phoneme vocabulary. The decoder exposes both a teacher-forcing `forward` (used during training) and an autoregressive `infer` method with greedy decoding (used during evaluation).

1.4 Cross-Attention

At decoder step t, raw alignment scores are computed as dot products between the current decoder hidden state and each encoder hidden state:

$$e_{t,i} = h_{t-1}^{\text{dec}} \cdot h_i^{\text{enc}}, \quad i = 1, \ldots, n, \tag{2}$$

normalised with softmax to attention weights $\alpha_{t,i}$, and used to form the weighted context vector $\mathbf{z}_t = \sum_i \alpha_{t,i}\, h_i^{\text{enc}}$.

1.5 Seq2Seq Wrapper

`Seq2Seq` composes `Encoder` and `Decoder` and exposes a unified `forward` that switches between teacher-forcing and autoregressive inference via a boolean flag.

2 Data Processing

2.1 Vocabularies

Both vocabularies are built exclusively from the training split to prevent data leakage. Four special tokens occupy the first four indices: `<PAD>`= 0, `<SOS>`= 1, `<EOS>`= 2, `<UNK>`= 3. The character vocabulary covers the 26 lowercase English letters (total 30 tokens); the phoneme vocabulary covers the 84 ARPAbet symbols (total 88 tokens).

Table 1: Dataset and vocabulary summary.

Split / Vocabulary	Size
Training words	92,426
Validation words	11,553
Test words	11,554
Character vocabulary	30
Phoneme vocabulary	88

2.2 Encoding and Padding

Each word is tokenised into characters (or space-separated phoneme tokens), truncated to a maximum of 16 positions (both source and target), prepended with `<SOS>` and appended with `<EOS>`, and then zero-padded to the fixed length. Padding to a uniform length within each batch is handled by PyTorch's default `DataLoader` collation because `Seq2SeqDataset` already returns fixed-length tensors.

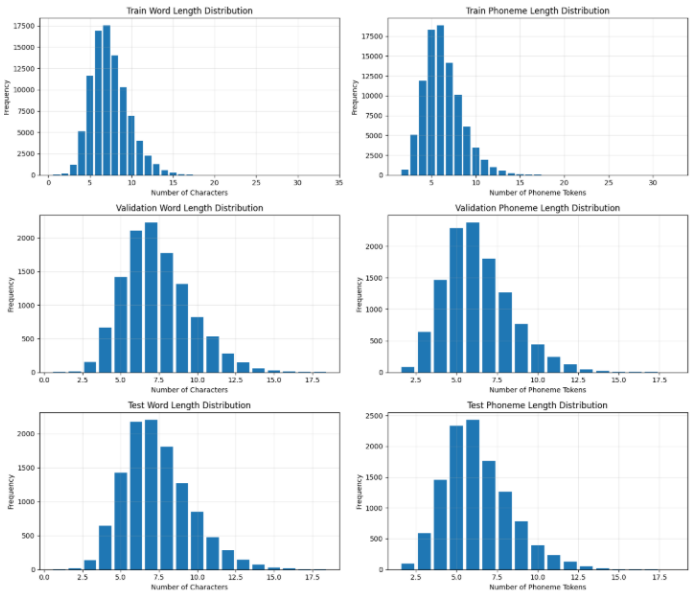

Figure 1: Word length (left) and phoneme sequence length (right) distributions for train, validation, and test splits.

3 Training

All models are trained with the **AdamW** optimiser and a cosine annealing learning-rate schedule (`CosineAnnealingLR`, $T_{\max}$ = epochs). The loss is cross-entropy with label smoothing $\epsilon = 0.1$ and `ignore_index=0` (`<PAD>`). Gradient clipping with max norm = 1.0 is applied at every step.

Stopping criterion. Training uses **early stopping** on validation *token accuracy* with patience $p = 5$ epochs, up to a maximum of 30 epochs. Token accuracy was chosen over validation loss because it is more directly tied to phoneme prediction quality. If accuracy does not improve for five consecutive epochs the run is terminated, saving compute while still allowing the model to recover from short plateaux.

4 Hyperparameter Tuning

Hyperparameter search was carried out on Setup 1 (no context vector) using **random search** over the Cartesian product {lr} × {embed} × {hidden}, with 10 randomly sampled trials each trained for 10 epochs. The search ranges were lr $\in \{10^{-5}, 10^{-4}, 10^{-3}\}$, embed dim $\in \{32, 128, 256\}$, and hidden size $\in \{64, 128, 256\}$. Validation *word accuracy* was used as the selection criterion.

Chosen configuration. The chosen configuration is lr = 10^{-3}, embed dim = 128, hidden size = 128.

Table 2: Hyperparameter search results (Setup 1, 10 epochs each). Best configuration highlighted in bold.

LR	Embed	Hidden	Val Word Acc
10^{-5}	128	256	0.15
10^{-3}	128	128	0.58
10^{-5}	32	256	0.07
10^{-3}	256	64	o.39
10^{-5}	128	256	0.15
10^{-4}	32	64	0.15
10^{-5}	128	256	0.14
10^{-5}	156	128	0.07
10^{-4}	256	128	0.28

5 Architecture Comparison

All three setups were trained under identical conditions: the same hyperparameters (Section 4), random seed 42, AdamW + cosine annealing, early stopping (patience 5, max 30 epochs), and batch size 16.

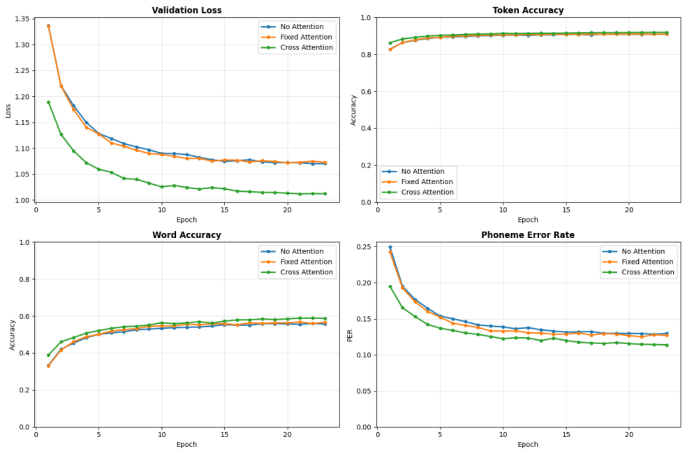

Figure 2: Validation loss, token accuracy, word accuracy, and PER for all three setups over training epochs.

Table 3: Test-set results for all three setups.

Setup	PER	Word Acc
Setup 1 — No context	0.134	0.559
Setup 2 — Fixed context	0.132	0.561
Setup 3 — Attention	0.120	0.577

Fixed context (Setup 2) vs. bottleneck (Setup 1). Fig. 2 and Table 3 show that Fixed context setup outperformed No attention or bottleneck setup(Setup 1) . This is because supplying the encoder's final hidden state $\mathbf{z} = h_n^{\text{enc}}$ at every decode step gives the decoder a persistent reminder of the global source representation. This relieves pressure on the initial hidden state and allows the decoder's own hidden

state to specialise in tracking output position rather than memorising all source information.

Attention (Setup 3) vs. fixed context (Setup 2). Similarly, cross-attention (Setup 3) outperformed Fixed-attention(Setup) because it recomputes a *different* context vector at each decode step by attending selectively to encoder positions. This is especially beneficial for longer words or irregular spellings where different output phonemes depend on non-adjacent input characters.

6 Evaluation

Phoneme Error Rate (PER) measures the average normalised edit distance between predicted and reference phoneme sequences. For a single word, PER = edit_distance$(\hat{y}, y)/|y|$, where each ARPAbet token (e.g. `EH1`) is treated as a single symbol. **Word accuracy** is the fraction of test words for which the entire predicted sequence exactly matches the reference. Both metrics are computed after stripping `<SOS>`, `<EOS>`, and `<PAD>` tokens from model output; the `nltk.metrics.distance.edit_distance` function is used for Levenshtein computation. Results are reported in Table 3.

7 Length-Bucket Analysis

To assess how the information bottleneck manifests with word length, test words are grouped into four buckets: 1–4, 5–7, 8–10, and 11+ characters. Word accuracy is computed separately in each bucket for all three setups.

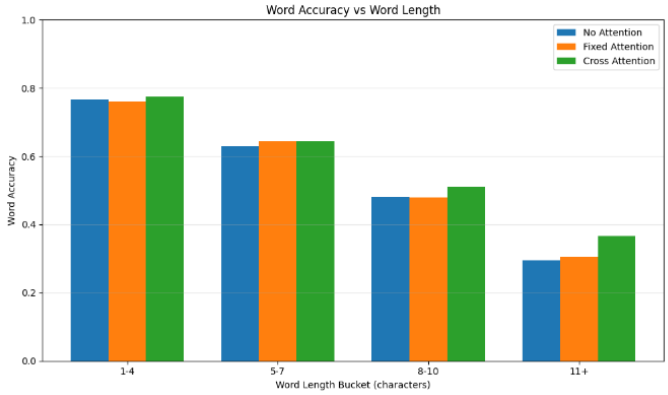

Figure 3: Word accuracy by source word-length bucket for all three setups.

Discussion. The bottleneck hypothesis predicts that Setup 1 (no context) should degrade *more steeply* with word length than Setups 2 and 3, because its fixed initial hidden state must compress increasing amounts of source information into the same fixed-size vector. Short words (1–4 chars) involve few source symbols, so the bottleneck is less severe; the three setups, can be seen in Fig .3, are performing comparably in this bucket as expected. For longer words (≥ 8 chars) the fixed context of Setup 2 provides a partial remedy by keeping the full encoding visible throughout decoding, while Setup 3's attention can focus on whichever encoder positions are most relevant to each output phoneme, mitigating degradation further.

8 Conclusion

This assignment implemented and compared three LSTM encoder-decoder variants for G2P conversion. The bottleneck baseline (Setup 1) constrains all source ninformation to the initial hidden state; the fixed-context variant (Setup 2) alleviates this by providing a persistent global encoding at every decode step; and the attention variant (Setup 3) further improves performance by computing a dynamic, position-sensitive context vector. The length-bucket analysis confirms that the performance gap between setups widens for longer and more irregularly spelled words, consistent with the bottleneck hypothesis.